Distinct-input affine stream panel: report

Question and scope

Does the full two-x affine table improve batch throughput over the ordinary affine-layer table on a stream of distinct scalar inputs? This retrospective report describes the frozen Tier 1 panel in this directory. It does not measure uniform random scalar inputs, one-scalar cold latency, or end-to-end application performance.

The fixture contains 12,288 unique scalars in 96 batches of 128. The generator verifies the 128 base scalars against secp256k1 curve equations and binary multiplication, derives a nonzero step from the base-fixture hash, and uses the scalar arithmetic progression induced by that step. The generated points are independently checked. The fixture SHA-256 is `6b7ce3b515d0980b9975398b8b31e5e1cfdc9e54510d04faf29fee3f3bacf778`.

Result

The frozen panel used 16 alternating repetitions on CPU 40 with adjacent A/A controls and the same binary and fixture. All 96 arms and 48 pairs completed, artifact hashes stayed stable, and every A/A ratio passed the predeclared `[0.90, 1.11]` gate. The independent audit is `tier1-affine-stream-cpu40-audit.json`; it checks the fixture, source and executable hashes, raw outputs, expected points, pair ordering, and gates. The raw receipts and provenance are archived in `1b1489f47213478b9aab096de03ce244.tar.gz` and `prime-j0-affine-stream-panel-provenance.tar.gz` with the digests recorded in the experiment README.

Metric	Ordinary affine-layer	Full two-x
Mean online time, 12,288 outputs	71.422170 ms	65.473654 ms
Mean time per scalar	5.812351 µs	5.328260 µs
Mean table preparation	3,966.995 ms	3,978.971 ms
Retained bytes	26,035,988	37,192,948

The predeclared case-specific median paired ordinary-affine/full-two-x ratio is **1.087393×**, with a paired bootstrap 95% interval [**1.074663, 1.106558**] over 16 comparison pairs. This is a qualified batch-throughput result for this arithmetic-progression fixture on the observed CPU 40 instance. Host exclusivity is unverified. It does not establish a gain for uniform random scalars or single-scalar latency. Table construction is measured separately and is slightly longer for full two-x in the observed means.

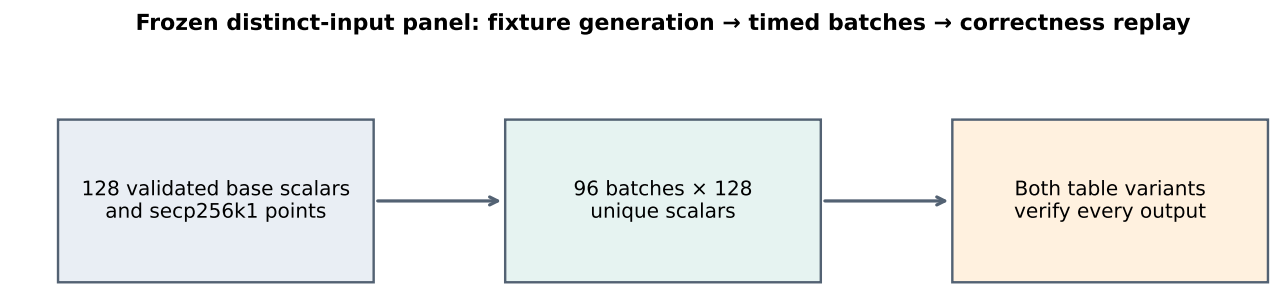
Protocol and evidence

The online interval covers fresh-input batches after fixture parsing and table construction; every output is checked after timing. The manifest and rules specify the repetitions, A/A limits, and artifact hashes. The generic runner's combined `paired_speedup=1.005872` is not the comparison estimate: it pools A/A controls with the comparison. The independent case-specific audit treats A/A as noise gates and computes the comparison interval separately.

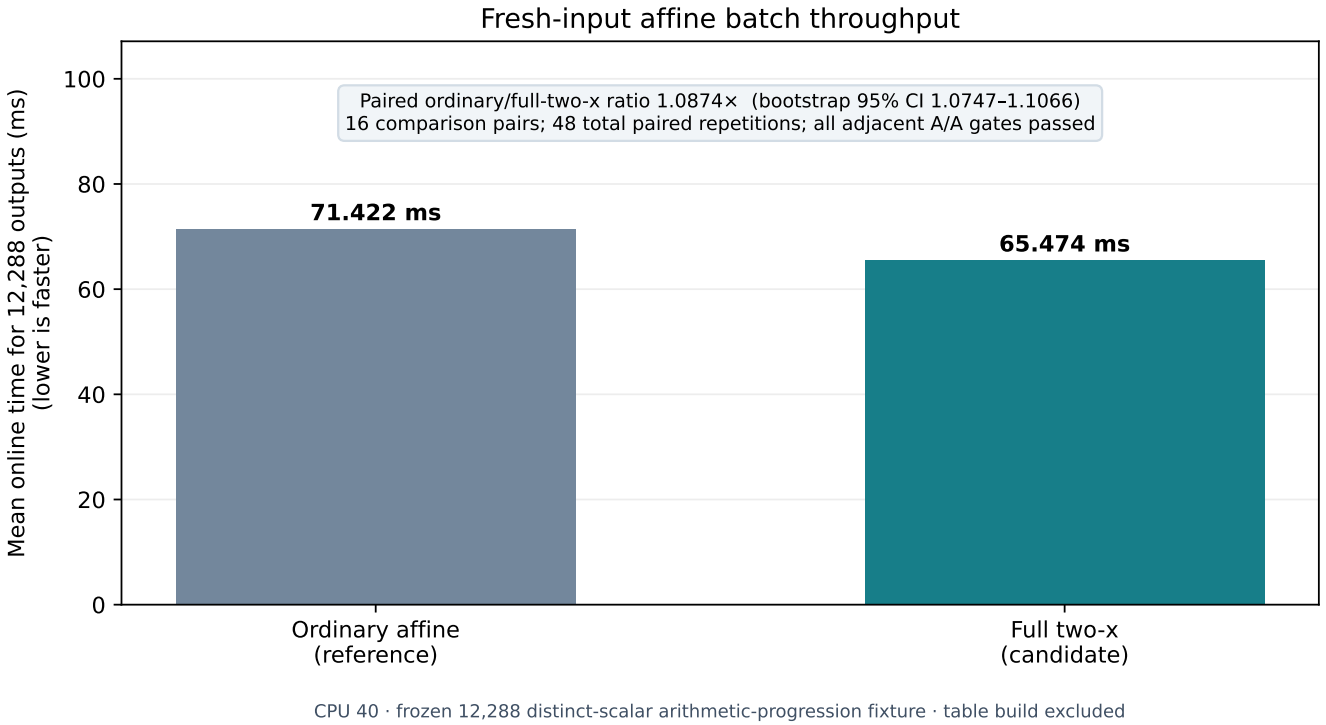
The diagram summarizes fixture generation and correctness replay. The throughput figure plots the audited means and paired interval. Source is `visuals.py`; it reads the committed audit JSON and writes the committed SVG and PDF figures. Rebuild with:

```
python3 experiments/prime-j0-affine-cache-budget-20261015/visuals.py
pandoc experiments/prime-j0-affine-cache-budget-20261015/REPORT-2026-10-19.md \
  --from=gfm --pdf-engine=xelatex -V geometry:margin=0.75in \
  -o experiments/prime-j0-affine-cache-budget-20261015/REPORT-2026-10-19.pdf
```

Fixture-flow SVG · PDF figure



Throughput SVG · PDF figure



Decision and limits

The panel supports retaining the full two-x table for this measured distinct-input batch workload. It does not support routing arbitrary workload classes through that variant without a workload-matched measurement. The next discriminating measurement is the already-separated uniform-scalar panel in #917; its input law and evidence must remain distinct from this arithmetic-progression result.